NEXUS — Manual técnico y preparación para el examen H3

Presentación y propósito

NEXUS v1.0 es un proyecto académico en C# ambientado en el año 2297. Un cadete explora una realidad desconocida, investiga una anomalía, recorre zonas, revisa su inventario, utiliza objetos, analiza entidades y busca completar una misión.

El objetivo de este manual es explicar el diseño del programa y servir como material de estudio para el Hito 3. La documentación relaciona los conceptos de Programación Orientada a Objetos (POO) con las clases conocidas del proyecto. Los ejemplos complementarios se identifican como tales y deben adaptarse si el código final difiere.

I. Arquitectura y programación orientada a objetos

Funcionamiento general

El punto de entrada está en Program.Main(). Según el código compartido, allí se crea un objeto SistemaNexus y se llama a nexus.Iniciar(). SistemaNexus coordina el menú y las acciones principales; las demás clases representan personajes, objetos, zonas, inventario y misión.

Responsabilidad de las clases identificadas

- Personaje: clase abstracta con información compartida, como Nombre y Vida, y un método virtual MostrarInformacion().
- Cadete: representa al jugador; mantiene estado como energía, experiencia e inventario y puede ganar experiencia.
- Robot y Enemigo: especializan a Personaje con información o comportamiento propios.
- Objeto: representa un elemento que puede guardarse y utilizarse en el inventario.
- Inventario: administra una colección de objetos y operaciones como mostrar, comprobar si está vacío, agregar y usar.
- Zona: representa una zona explorable.
- Mision: administra información de la misión actual.
- SistemaNexus: coordina la ejecución y acciones como iniciar, usar objeto, analizar una entidad y completar la misión.
- Program: contiene Main(), el inicio de la aplicación.

Diagrama de clases y relaciones

En el diagrama preparado se representan las clases principales y las relaciones conocidas. Personaje es una clase base abstracta; Cadete, Robot y Enemigo heredan de ella. Cadete mantiene un Inventario, y el Inventario administra objetos de tipo Objeto. SistemaNexus coordina el uso de estas clases y se relaciona con Zona y Mision.

Revisa el diagrama contra el código completo antes de entregar: las flechas de herencia deben apuntar hacia Personaje; una relación de composición o asociación debe reflejar lo que realmente construye o mantiene cada clase.

Programación estructurada frente a POO

La programación estructurada organiza la solución principalmente mediante instrucciones, secuencias, condiciones y ciclos. La POO organiza el programa alrededor de clases y objetos que agrupan datos y comportamientos.

En NEXUS, las clases Cadete, Inventario y Objeto separan responsabilidades. Esto permite modificar la lógica del inventario sin tener que concentrar todos sus detalles en Main(). La POO no elimina las condiciones o ciclos: los utiliza dentro de los métodos de los objetos.

II. Fundamentos de clases y objetos

Clases, objetos e instanciación

Una clase es una definición que describe datos y comportamientos. Un objeto es una instancia concreta de esa clase. Por ejemplo, la instrucción SistemaNexus nexus = new SistemaNexus(); crea un objeto y guarda una referencia en la variable nexus.

Campos, atributos, propiedades y métodos

Los campos almacenan valores directamente. Las propiedades ofrecen una forma controlada de leer o modificar datos. Los métodos ejecutan acciones. En Personaje, Nombre y Vida aparecen como propiedades; MostrarInformacion() es un método.

Constructores y sobrecarga

Un constructor se ejecuta al crear el objeto y suele inicializar su estado. En el código compartido, Personaje recibe nombre y vida mediante Personaje(string nombre, int vida). La sobrecarga consiste en declarar varios constructores o métodos con el mismo nombre y distintos parámetros. No se debe afirmar que existe sobrecarga en una clase si no se comprueba en el código completo.

Encapsulamiento y modificadores

El encapsulamiento busca proteger el estado interno y ofrecer acceso controlado. En public string Nombre { get; protected set; }, get permite leer la propiedad y protected set permite cambiarla desde la clase que la declara o sus clases derivadas, pero no desde cualquier código externo.

Uso de this y base

this hace referencia al objeto actual y puede distinguir una propiedad de un parámetro con el mismo nombre. base permite acceder a miembros de la clase base, por ejemplo llamar al constructor de Personaje desde el constructor de Cadete. Verifica las llamadas exactas en tus constructores.

III. Herencia, polimorfismo e interfaces

Herencia y especialización

La herencia permite que una clase derivada reutilice miembros de una clase base y añada o especialice comportamiento. En NEXUS, Personaje es abstracta y Cadete, Robot y Enemigo derivan de ella.

virtual, override y abstract

Un método virtual puede redefinirse en una clase derivada. override declara una implementación que reemplaza el comportamiento virtual heredado. Una clase abstracta no se instancia directamente y puede declarar métodos abstractos que las clases derivadas deben implementar. En el código compartido, Personaje es abstracta y MostrarInformacion() aparece como virtual; comprueba qué clases usan override.

Polimorfismo

El polimorfismo permite tratar objetos de clases derivadas mediante una referencia del tipo base y ejecutar la implementación correspondiente al objeto real. En AnalizarEntidad(Personaje personaje), el parámetro usa el tipo Personaje y luego se llama a personaje.MostrarInformacion(). Si las clases derivadas sobrescriben el método, se ejecuta la versión correspondiente al objeto.

Interfaces: ejemplo complementario

En los fragmentos compartidos no hay evidencia suficiente para afirmar que NEXUS ya implementa interfaces. Una interfaz define un contrato que una clase se compromete a cumplir. Ejemplo complementario:

```
public interface IAnalizable
{
    void Analizar();
}

public class Robot : Personaje, IAnalizable
{
    public void Analizar()
    {
        Console.WriteLine("Analizando robot...");
    }
}
```

Este fragmento es ilustrativo y no debe copiarse sin adaptarlo a los constructores y miembros reales de Robot. Herencia expresa una relación de especialización; composición expresa que un objeto contiene o administra otro; una interfaz expresa un contrato de operaciones.

IV. Colecciones y LINQ

List<T> y recorrido de objetos

Una List<T> es una colección genérica de elementos de tipo T. En el inventario de NEXUS, la idea es mantener objetos y operar sobre ellos. Comprueba el tipo y el nombre exactos de la lista en Inventario.cs o en el archivo donde esté declarada.

Recorrido tradicional

```
// Ejemplo complementario: recorrer una lista de objetos
foreach (Objeto objeto in objetos)
{
    Console.WriteLine(objeto.Nombre);
}
```

LINQ y expresiones lambda

No hay evidencia suficiente en los fragmentos compartidos para afirmar que el proyecto usa LINQ. LINQ permite consultar colecciones; una lambda es una expresión breve que define una función. Ejemplo complementario para filtrar objetos por nombre:

```
// Ejemplo complementario; requiere que Objeto tenga una propiedad Nombre
var resultados = objetos
    .Where(o => o.Nombre.Contains("energia"))
    .OrderBy(o => o.Nombre)
    .Select(o => o.Nombre);
```

Where filtra elementos, OrderBy los ordena y Select transforma cada elemento en un resultado. Para usar LINQ normalmente se necesita using System.Linq;. El ejemplo debe ajustarse al nombre real de la propiedad y a las reglas del proyecto.

LINQ frente a recorridos tradicionales

Un foreach suele ser claro para ejecutar acciones sobre cada elemento. LINQ resulta útil para expresar filtros, ordenamientos y transformaciones de forma declarativa. No siempre es necesario usar LINQ; se elige según la claridad de la solución.

V. Análisis del funcionamiento

Flujograma de UsarObjeto()

El flujograma creado representa el proceso observado en el código compartido: limpiar pantalla, mostrar inventario, comprobar si está vacío, solicitar un número, validar la entrada y llamar a cadete.Inventario.Usar(numero, cadete). Incluye las salidas para inventario vacío y entrada inválida. Inserta aquí la imagen Diagrama_Flujo_UsarObjeto_NEXUS.png.

Seguimiento del estado de un objeto

Ejemplo ilustrativo basado en el diseño compartido: al iniciar, el cadete tiene un valor inicial de energía y un inventario con objetos. Cuando se usa una célula de energía, el método de inventario entrega el objeto al cadete y, si su implementación lo define así, aumenta la energía. El valor exacto final depende de las validaciones y límites implementados en el método de uso; compruébalo en el código antes de afirmar un resultado numérico.

Predicción de código

```
Personaje personaje = new Robot(/* argumentos reales del constructor */);
personaje.MostrarInformacion();
```

Predicción razonada: la variable tiene tipo declarado Personaje, pero referencia un objeto Robot. Si Robot sobrescribe MostrarInformacion(), se ejecutará la versión de Robot. El ejemplo es conceptual; sustituye los argumentos por el constructor real de tu proyecto.

Qué ocurre si se modifica una condición

En UsarObjeto(), la entrada se valida con int.TryParse y se comprueba que el número sea mayor que cero. Si se elimina la validación, una entrada como abc podría provocar un fallo al convertirla. Si se elimina la condición numero > 0, podrían llegar valores no válidos al método Inventario.Usar y su comportamiento dependería de sus propias comprobaciones.

VI. Preparación para el examen: preguntas razonadas

1. ¿Por qué Personaje se define como clase abstracta?

Porque representa características comunes y sirve como base para tipos concretos. No se crea directamente un Personaje; se crean instancias de clases derivadas como Cadete, Robot o Enemigo.

2. ¿Cuál es la diferencia entre una clase y un objeto?

La clase describe la estructura y el comportamiento; el objeto es una instancia concreta creada a partir de esa definición. SistemaNexus nexus = new SistemaNexus(); crea una instancia.

3. ¿Qué aporta el encapsulamiento en Nombre { get; protected set; }?

Permite leer Nombre desde código que tenga acceso a la propiedad, pero restringe su modificación a la clase que la declara y sus derivadas. Esto reduce cambios arbitrarios desde código externo.

4. ¿Cómo se observa el polimorfismo en AnalizarEntidad(Personaje personaje)?

El método recibe una referencia de tipo Personaje y llama a MostrarInformacion(). Si el objeto real es un Robot o un Enemigo y su clase sobrescribe el método, se ejecuta la implementación correspondiente al tipo real.

5. ¿Qué diferencia hay entre herencia, composición e interfaz?

La herencia especializa una clase base; la composición representa que un objeto contiene o administra otro objeto; una interfaz establece un contrato de operaciones que una clase implementa. El inventario del Cadete ilustra una relación de tipo 'tiene un'.

6. ¿Qué sucede si el usuario escribe abc en UsarObjeto()?

int.TryParse debería devolver false, por lo que el programa muestra un mensaje de entrada inválida y no debe intentar usar el número. Esto depende de que se conserve la validación del código compartido.

7. ¿Para qué sirven List<T> y LINQ?

List<T> almacena una colección de elementos de un tipo determinado. LINQ permite filtrar, ordenar o transformar colecciones. En este manual LINQ se presenta como concepto complementario porque no se confirmó su uso en los fragmentos compartidos.

8. ¿Qué diferencia hay entre virtual y override?

virtual permite que una clase derivada cambie el comportamiento de un método; override declara la implementación que sustituye el método virtual heredado.

9. ¿Qué riesgo hay al quitar la validación numero > 0?

Podrían enviarse números cero o negativos al método de inventario. Si ese método no los controla, podría seleccionar mal un elemento o generar un comportamiento incorrecto.

10. ¿Cómo comprobarías que un diagrama es correcto?

Compararía cada clase, propiedad, método y relación con el código fuente. Confirmaría las flechas de herencia, las asociaciones reales y los nombres exactos, y corregiría cualquier elemento inventado por la herramienta.

Diagnóstico personal de aprendizaje — borrador

El estudiante avanzó paso a paso en la publicación del repositorio, creó README.md y docs/README.md, compartió fragmentos de código y elaboró diagramas. Estas acciones muestran participación en la organización y documentación del proyecto.

La conversación no demuestra por sí sola dominio independiente de todos los conceptos de POO. La evidencia sobre interfaces y LINQ es insuficiente, y es necesario practicar predicción de salidas, explicación de relaciones entre clases y modificación de métodos sin ayuda.

Prioridades sugeridas: explicar herencia y polimorfismo usando Personaje; distinguir propiedades y encapsulamiento; trazar las validaciones de UsarObjeto; practicar List<Objeto> y LINQ; diseñar y explicar una interfaz complementaria.

Este texto es un diagnóstico provisional basado en la conversación y no es el resultado íntegro del PROMPT ESPEJO solicitado por la consigna. Debe reemplazarse o complementarse con el resultado generado mediante el prompt exacto al finalizar el manual.

Lista de verificación antes de entregar

- Verificar el código completo y compilar el proyecto en Visual Studio.
- Comprobar que nombres, propiedades, métodos y constructores coincidan con Programa.cs y los demás archivos .cs.
- Insertar y revisar los diagramas de clases y de flujo.
- Verificar los ejemplos complementarios de interfaces y LINQ; no presentarlos como funciones ya implementadas.
- Generar el resultado íntegro del PROMPT ESPEJO al final del manual.
- Guardar el manual en docs/ y exportarlo a PDF.
- Revisar que .vs/, bin/, obj/ y datos sensibles no estén publicados.

PROMPT ESPEJO — texto que debe enviarse al finalizar

La consigna indica que, cuando el manual esté terminado, se debe enviar exactamente el prompt siguiente en el mismo chat. Después, se debe incorporar íntegramente su resultado al final del documento.

```
Actúa como un evaluador de mi proceso de aprendizaje durante la misión NEXUS: DE CÓDIGO A CONOCIMIENTO.

Analiza nuestra conversación completa, prestando especial atención a mis aportes personales: preguntas, razonamientos, explicaciones, decisiones, errores y correcciones.

Genera un diagnóstico con estos apartados:

1. Fortalezas demostradas: identifica los conceptos que expliqué correctamente y las evidencias de comprensión que aporté.

2. Falencias observadas: identifica errores conceptuales, dificultades de razonamiento, dudas recurrentes y explicaciones incompletas.

3. Diagnóstico del Hito 3: analiza POO, clases y objetos, constructores, encapsulamiento, herencia, polimorfismo, interfaces, listas genéricas y LINQ. Clasifica cada concepto como comprensión demostrada, comprensión parcial, dificultad observable o evidencia insuficiente. Justifica tus conclusiones.

4. Mi forma de aprender: evalúa cómo investigué, contrasté resultados, resolví dudas y corregí errores.

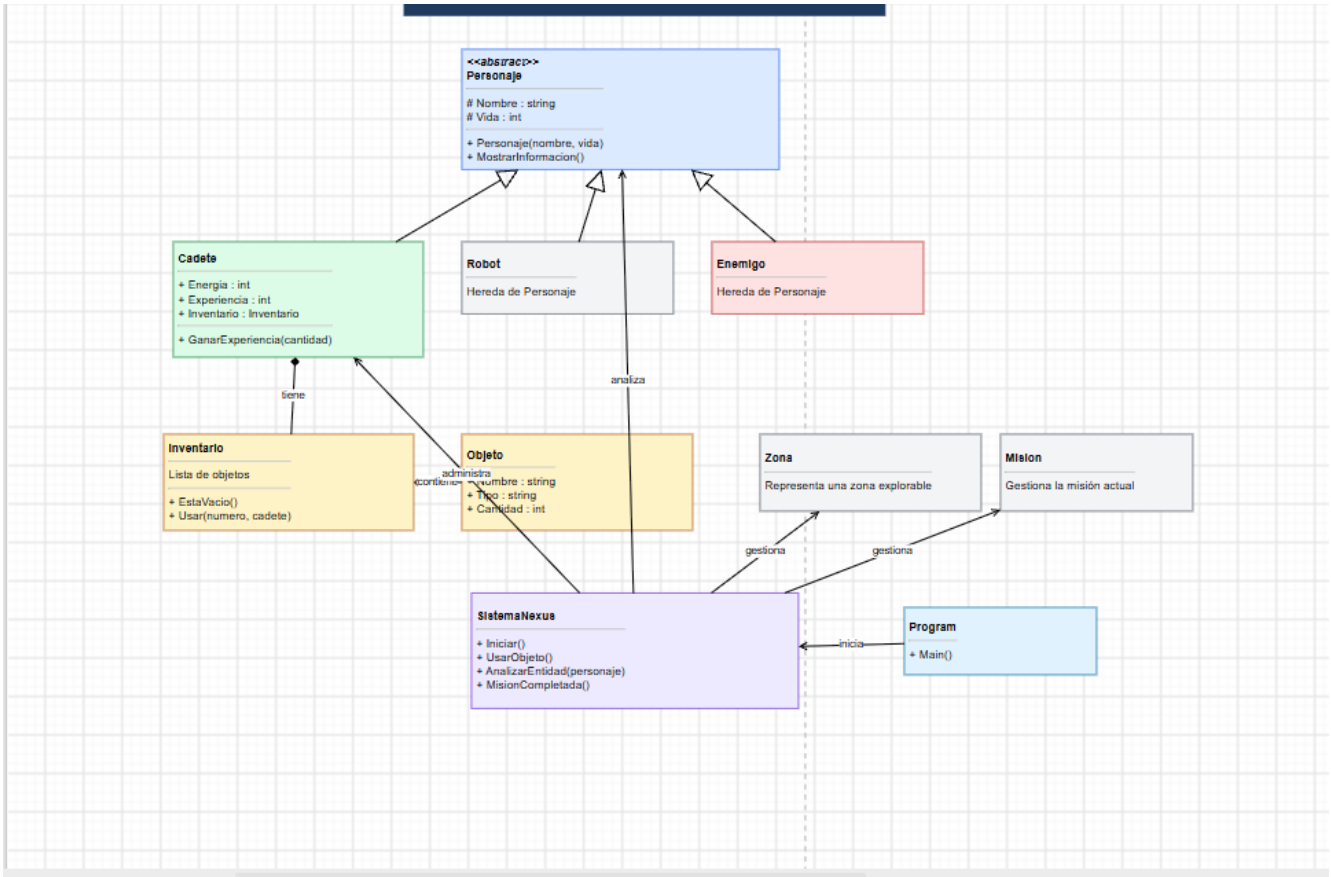
5. Plan de estudio: identifica los cinco conocimientos que debo reforzar con mayor prioridad antes del examen. Propón un ejercicio práctico para cada uno.

6. Conclusión: explica qué conocimientos he demostrado, cuáles necesitan refuerzo y qué debería poder resolver sin ayuda.

No atribuyas a mis capacidades las respuestas que tú generaste. No inventes evidencias. Si no
```

hay información suficiente para evaluar un concepto, indícalo. Redacta un diagnóstico concreto, crítico, respetuoso y basado exclusivamente en la conversación.




<<abstract>>
Personaje
Nombre : string
Vida : int
+ Personaje(nombre, vida)
+ MostrarInformacion()
Cadete
+ Energia : int
+ Experiencia : int
+ Inventario : Inventario
+ GanarExperiencia(cantidad)
Robot
Hereda de Personaje
Enemigo
Hereda de Personaje
tiene
analiza
Inventario
Lista de objetos
+ EstaVacio()
+ Usar(numero, cadete)
administra
Objeto
+ Nombre : string
+ Tipo : string
+ Cantidad : int
Zona
Representa una zona explorable
Mision
Gestiona la misión actual
gestiona
gestiona
SistemaNexus
+ Iniciar()
+ UsarObjeto()
+ AnalizarEntidad(personaje)
+ MisionCompletada()
inicia
Program
+ Main()

Diagnóstico personal de aprendizaje — NEXUS

1. Fortalezas demostradas

- Logré avanzar en la publicación del proyecto NEXUS en GitHub.
- Seguí los pasos para organizar la documentación en una carpeta del repositorio.
- Compartí código para analizar la estructura y el funcionamiento del programa.
- Utilicé draw.io para trabajar con los diagramas y aprendí a exportarlos como imágenes.
- Identifiqué las tareas necesarias para preparar el manual técnico.

Estas evidencias demuestran que pude avanzar en la organización y documentación del proyecto con orientación paso a paso. No demuestran por sí solas que domine todos los conceptos de programación orientada a objetos.

2. Falencias observadas

Durante el proceso necesité ayuda frecuente para realizar operaciones en GitHub, guardar y exportar archivos y organizar los diagramas. Esto indica que debo practicar más el manejo independiente de estas herramientas.

También necesito reforzar la explicación del código con mis propias palabras. La conversación no contiene suficientes explicaciones personales para comprobar que domino todos los conceptos de POO.

3. Diagnóstico del Hito 3

Concepto	**Evaluación**
Clases y objetos	Comprensión parcial: trabajé con el código, pero falta demostrar una explicación independiente.
Constructores	Evidencia insuficiente para evaluar el dominio.
Encapsulamiento	Evidencia insuficiente para confirmar comprensión independiente.
Herencia	Comprensión parcial: se identificaron clases derivadas de Personaje.
Polimorfismo	Evidencia insuficiente para comprobar que puedo predecir llamadas a métodos sin ayuda.
Interfaces	Evidencia insuficiente; el ejemplo de IAnalizable es complementario.
Listas genéricas	Comprensión parcial: se revisó el uso del inventario y los objetos.
LINQ	Evidencia insuficiente para demostrar su aplicación independiente.

4. Mi forma de aprender

Trabajé mediante instrucciones paso a paso, capturas de pantalla y correcciones sucesivas. Cuando aparecieron dificultades, compartí lo que veía para continuar el proceso.

Mi siguiente objetivo es intentar resolver tareas sencillas sin depender de instrucciones para cada clic y explicar qué hace cada parte del código antes de modificarla.

5. Plan de estudio prioritario

1. Clases y objetos: crear un objeto Cadete y explicar sus propiedades y métodos.
2. Encapsulamiento y constructores: explicar qué hace get, set, protected y el constructor.
3. Herencia y polimorfismo: comparar Cadete, Robot y Enemigo, y predecir qué método se ejecuta.
4. Listas y LINQ: filtrar una lista de objetos con Where y explicar el resultado.
5. Análisis de código: seguir las validaciones de UsarObjeto() con las entradas abc, 0 y 1.

6. Conclusión

He demostrado avances en la publicación, organización y documentación de NEXUS, además de perseverancia al resolver dificultades técnicas. Sin embargo, necesito practicar la explicación y modificación independiente del código para demostrar dominio de los conceptos del Hito 3.

Antes del examen debería poder explicar las relaciones entre las clases, predecir la salida de fragmentos de código, identificar errores y justificar cómo una modificación afecta al funcionamiento del programa.